

Automata and Formal Languages

Dr. Sankhadeep Chatterjee

Closure Properties of Recursive and RE Languages

Operation	Recursive (Decidable)	Recursively Enumerable (Turing Recognizable)
Union	Yes	Yes
Concatenation	Yes	Yes
Star	Yes	Yes
Intersection	Yes	Yes
Complement	Yes	No

Proof of Closure Under Union

- Let L_1 and L_2 be two decidable languages.
- By definition, there exist halting Turing machines M_1 and M_2 that decide L_1 and L_2 , respectively. To prove $L_1 \cup L_2$ is decidable, we design a new Turing machine M as follows:
- Input: A string x
- Algorithm:
 - Simulate M_1 on x . If M_1 accepts, then M accepts.
 - If M_1 does not accept, simulate M_2 on x . If M_2 accepts, then M accepts; otherwise, M rejects
- Correctness:
 - Because M_1 and M_2 are halting machines, M is guaranteed to always halt.
 - If x belongs to either L_1 or L_2 , M will accept it;
 - if x belongs to neither, M will reject it. Thus, decidable languages are closed under union

Church-Turing thesis

- The Church-Turing thesis theorizes that any problem that can be computed by any computational device can also be computed by a Turing machine, provided it is represented in a suitable language format
- This implies that Turing machines serve as a universal model of computation
- Consequently, if a problem is shown to be undecidable, it signifies that the problem cannot be computed by any existing or hypothetical computing device

Enumeration and Representation of Turing Machines

- Every Turing machine (TM) can be represented as a finite string over a finite alphabet, describing its transition function, states, and accept/reject conditions
- Binary Conversion: To standardize these descriptions, the string is converted into a binary string over $\{0, 1\}$. Each symbol from the original alphabet of size k is replaced by a unique binary string of length $\lceil \log k \rceil$
- Mapping to Natural Numbers: To ensure every binary string maps to a unique natural number, a '1' is appended to the beginning of the string to prevent different strings with preceding zeros from mapping to the same decimal value .

Enumeration and Representation of Turing Machines

- Onto Function: An onto function f is established from the set of natural numbers to the set of all Turing machines. This ensures every Turing machine has at least one corresponding natural number (pre-image).
- Natural numbers that do not naturally represent a valid machine are mapped to a fixed default machine, such as one that accepts all inputs
- Notation: M_j denotes the j^{th} Turing machine in this enumeration, and s_j denotes the j^{th} binary string.

The Diagonalization Language (L_d)

- The language L_d is defined as the set of all binary strings s_j such that s_j does not belong to the language of M_j ($s_j \notin L(M_j)$).
- A semi-infinite matrix O is constructed where rows represent machines (M_i) and columns represent strings (s_j). The entry $O(i, j) = 1$ if M_i accepts s_j , and 0 otherwise.
- L_d is created by examining the diagonal entries $O(j, j)$. If the j^{th} machine does not accept the j^{th} string ($O(j, j) = 0$), s_j is included in L_d .

		All possible binary strings				
		s_1	s_2	s_3	...	s_j
All Turing Machines	M_1	0	1	0	...	0
	M_2	1	1	1	...	1
	M_3	0	1	0	...	1
	$\vdots$	$\vdots$	$\vdots$	$\vdots$		$\vdots$
	M_i	0	1	1	...	Diagonal

The Diagonalization Language (L_d) is Non-RE

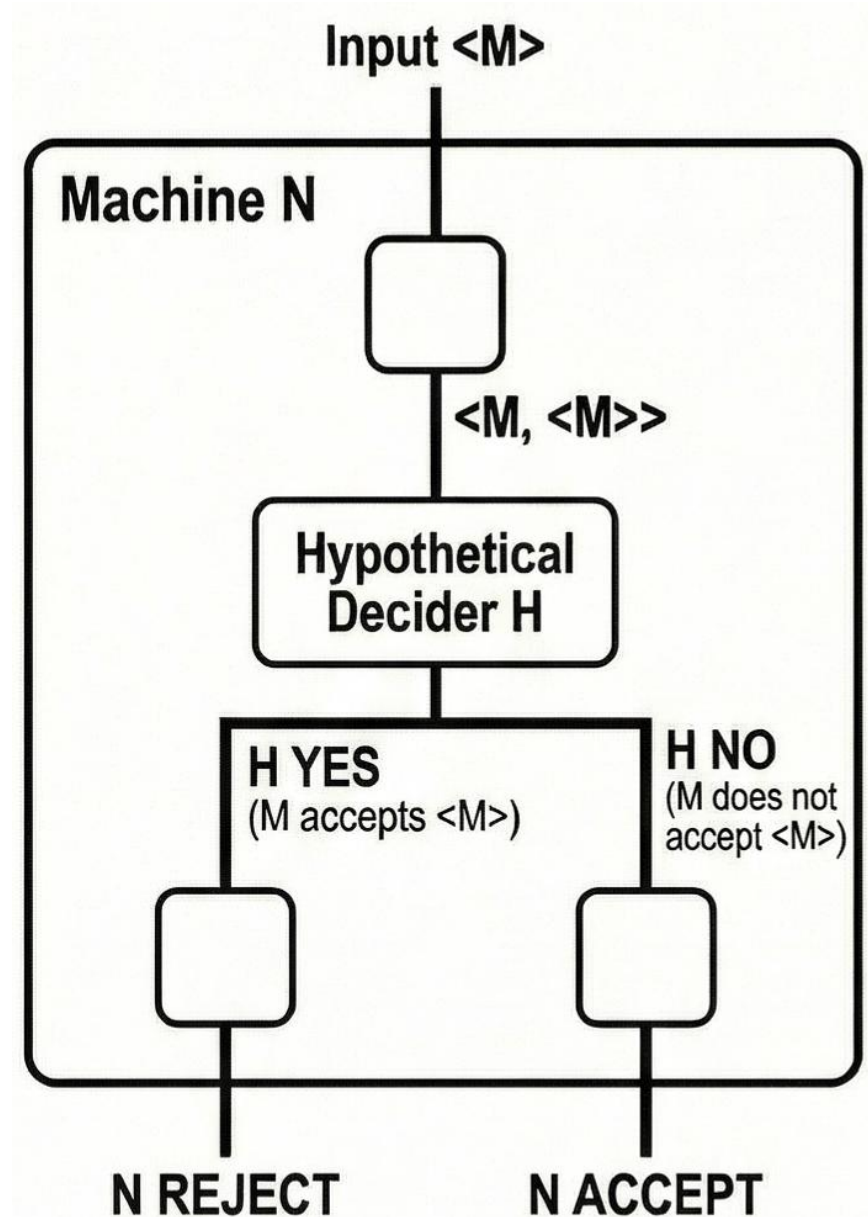
- Suppose L_d were $L(M)$ for some TM M .
- Since L_d is a language over alphabet $\{0,1\}$, M would be in the list of Turing machines we have constructed, since it includes all TM's with input alphabet $\{0,1\}$.
- Thus, there is at least one code for M , say i ; that is, $M = M_i$.
 - If $w_i \in L_d$, then M_i accepts w_i . But then, by definition of L_d , $w_i \notin L_d$, because L_d contains only those w_j such that M_j does not accept w_j .
 - Similarly, if $w_i \notin L_d$, then M_i does not accept w_i . Thus, by definition of L_d , $w_i \in L_d$.
- Since w_i can neither be in L_d nor fail to be in L_d , we conclude that there is a contradiction of our assumption that M exists.
- Hence, L_d is not a recursively enumerable language.

Recursively Enumerable but not Recursive

- *The Acceptance Problem (A_{TM}):* A_{TM} consists of all pairs $\langle M, w \rangle$ such that the Turing machine M accepts string w
- *The Halting Problem (H_{TM}):* H_{TM} is the set of pairs $\langle M, w \rangle$ such that M halts on w (either accepts or rejects)

The Acceptance Problem (A_{TM})

- **Hypothetical Decider (H):** We start by assuming ATM is decidable. This implies there exists a halting Turing machine, H and given an input pair $\langle M, w \rangle$, H outputs **YES** if M accepts w and **NO** if M does not accept w .
- **Construction of N :** We design a new machine, N , that has the code of H hardwired into it.
 - **Input:** N takes the encoding of a machine $\langle M \rangle$ as its input.
 - **Process:** N runs H on the input pair $\langle M, \langle M \rangle \rangle$.
 - **Reversal Logic:** N is designed to do the opposite of H : if H says M accepts its own encoding, N **rejects**; if H says M does not accept its own encoding, N **accepts**.
- **The Contradiction:** When N is fed its own encoding $\langle N \rangle$, N accepts $\langle N \rangle$ if and only if N does not accept $\langle N \rangle$



The Halting Problem (H_{TM})

- We assume H_{TM} is decidable by some machine N . We then use N as a component inside a larger machine, H , designed to decide A_{TM} .
- **Algorithm:**
 - **Step 1:** The machine H receives $\langle M, w \rangle$. It first gives this pair to the "black box" N to ask, "Does machine M halt on w ?".
 - **Step 2:**
 - If N outputs **NO**, then M must loop forever on w . H can safely **reject** without getting stuck in a loop.
 - If N outputs **YES**, H knows M is guaranteed to halt. H then simulates M on w to see if it accepts or rejects, and outputs that final answer.
- Since H would be a decider for the undecidable ATM , the "black box" N for the Halting Problem cannot exist.

